

정렬 비교 보고서 — 삽입 · 병합 · Gnome

작성자: 변정우

구현: src/sort.c (../src/sort.c) · src/main.c (../src/main.c) · src/sort.h (../src/sort.h)

재현: make test · make run > report/results.csv · python3 report/make_charts.py

깃허브주소: <https://github.com/soulkingjw9642-tech/Compare-sorting/tree/main>

차례

1. 코드 보고서 : 파일 구성, 공통 인터페이스, 검증
2. 알고리즘 상세: 세 알고리즘의 작동 원리와 복잡도
3. 알고리즘 실험: 조건, 결과, 그래프, 해석과 한계

1. 코드 보고서

1.1 비교의 기준과 인터페이스

코드는 `Item { int key; int original; }` 배열을 정렬한다. `key` 만 비교하고 `original` 은 정렬 전 위치를 보존하므로 동일 키의 상대 순서가 바뀌었는지 검사할 수 있다. 세 정렬 함수는 모두 `Item *a, size_t n, Stats *s` 를 인자로 받는다. `Stats` 는 비교와 이동 횟수를 담는다.

```
/* src/sort.h */
typedef struct { int key; int original; } Item;
typedef struct { unsigned long long comparisons, moves; } Stats;
typedef void (*SortFn)(Item *, size_t, Stats *);
void insertionSort(Item *, size_t, Stats *);
void mergeSort(Item *, size_t, Stats *);
void gnomeSort(Item *, size_t, Stats *);
```

샘플 보고서는 일반적인 `void *` 비교 인터페이스를 사용한다. 이 과제에서는 정수 키와 원본 순서를 가진 고정된 `Item` 으로 범위를 한정해 코드를 간결하게 유지했다. 따라서 다른 자료형에 즉시 적용하는 범용 정렬 라이브러리는 아니다. `main.c` 의 `SortFn` 배열을 통해 같은 호출 루프에서 세 함수를 차례로 실행한다.

1.2 파일 구조와 자료 흐름

그림 1 코드 구조와 자료 흐름

tests/test_sort.c 검사	→ src/sort.h 공통 규약	← src/main.c 입력과 측정
insertionSort	mergeSort	gnomeSort
	↓ results.csv 48 행	↓ 그래프와 보고서

파일	역할	확인할 부분
src/sort.h	공통 데이터와 세 함수 선언	Item, Stats, SortFn
src/sort.c	세 정렬과 공통 비교·대입 계수	greater, assign, mergeRec, gnomeSort

파일	역할	확인할 부분
src/main.c	고정 시드 입력 생성, 반복 시간 측정, 정렬 및 안정성 관찰	input, 정렬 호출 루프, CSV 출력
tests/test_sort.c	빈 배열, 중복, 역순, 음수 포함 기초 검사	3 정렬 × 4 입력 묶음
report/results.csv	실험 48 행 원본	4 형태 × 4 크기 × 3 정렬
report/make_charts.py	CSV 에서 그래프 생성	값 재측정 없이 그림 갱신
report/make_pdf.py	PDF 생성	포함된 표·그림과 본문

측정 대상 입력은 먼저 source 에 만든다. 각 정렬·반복 직전에 memcpy 로 work 를 복구하고 정렬 호출 직전과 직후의 clock() 차이만 시간에 더한다. 정렬 이후의 오름차순 검증도 측정 구간 밖에서 이루어진다. 따라서 같은 표 안의 세 알고리즘은 같은 값의 배열을 받는다.

1.3 비교·이동의 정의

greater(x, y, s) 호출 한 번을 키 비교 1 회, assign(dst, src, s) 호출 한 번을 원소 이동 1 회로 기록한다. 원소를 지역변수에 잠시 저장하는 대입과 입력 복사 memcpy 는 이동 수에 포함하지 않는다. 이는 실제 기계 명령 수나 메모리 접근 총량이 아니라 이 코드에서 정의한 작업 횟수다. 삽입 정렬은 큰 값을 오른쪽으로 옮길 때, 병합 정렬은 임시 배열에 쓰고 다시 원본에 복사할 때, Gnome sort 는 교환 과정의 두 배열 대입을 센다.

1.4 정확성 확인과 범위

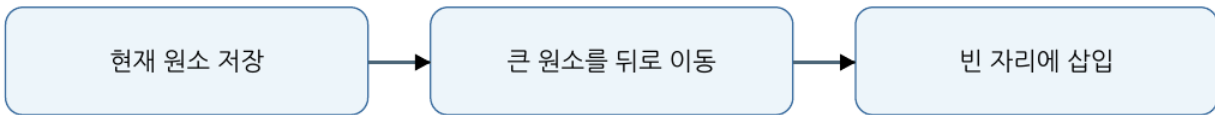
make test 는 3 개 함수에 각각 4 개 기초 입력 묶음을 적용해 12 개 알고리즘·입력 조합이 통과하는지 확인한다. 모든 정렬에서 키의 비내림차순을 확인하고, 세 정렬 모두 같은 키의 original 이 증가하는지도 검사한다. make run 역시 매 정렬 뒤 전체 배열을 훑어 키 순서 오류가 있으면 비정상 종료한다. 이 검사는 대표 사례의 통과를 뜻하며 모든 가능한 입력에 대한 형식적 증명은 아니다.

```
make test
make run > report/results.csv
python3 report/make_charts.py
python3 report/make_pdf.py
```

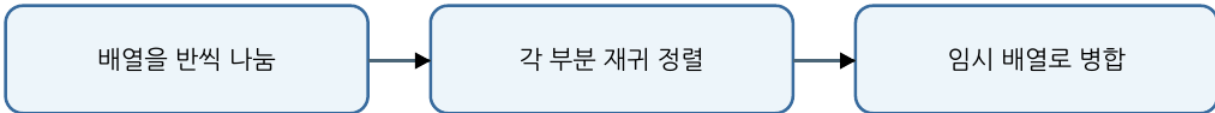
2. 알고리즘 상세

2.1 삽입 정렬: 정렬된 앞부분에 끼워 넣기

왼쪽 a[0..i-1]이 이미 정렬됐다고 가정하고, a[i]를 임시로 보관한다. 왼쪽에서 현재 값보다 큰 원소를 한 칸씩 오른쪽으로 옮긴 뒤 빈자리에 넣는다. 큰지 판단할 때 >를 사용하므로 같은 키는 앞질러 지나가지 않는다.



병합 정렬



Gnome sort: 새로 학습한 알고리즘

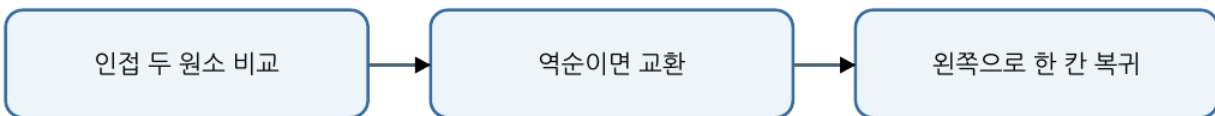


그림 2 세 정렬의 동작 흐름

예: [4, 2, 3, 1]에서 2를 삽입하면 [2, 4, 3, 1], 이어 3을 삽입하면 [2, 3, 4, 1], 마지막으로 1을 삽입하면 [1, 2, 3, 4]다. 이미 정렬된 입력은 각 i 마다 한 번 비교하고 옮기지 않으므로 $O(n)$ 이다. 역순에서는 앞의 모든 원소를 밀어 대략 $n(n-1)/2$ 번 비교·이동하므로 $O(n^2)$ 이다. 별도의 배열은 쓰지 않는다.

2.2 병합 정렬: 분할하고 정렬된 부분을 합치기

`mergeRec(a, tmp, lo, hi, s)`는 구간을 반으로 나눠 각각 정렬한 뒤, 왼쪽과 오른쪽의 현재 원소 중 작은 것을 임시 배열에 쓴다. 동물이면 왼쪽을 먼저 선택해 기존 상대 순서를 지킨다. 병합한 구간은 원본 배열로 복사한다.

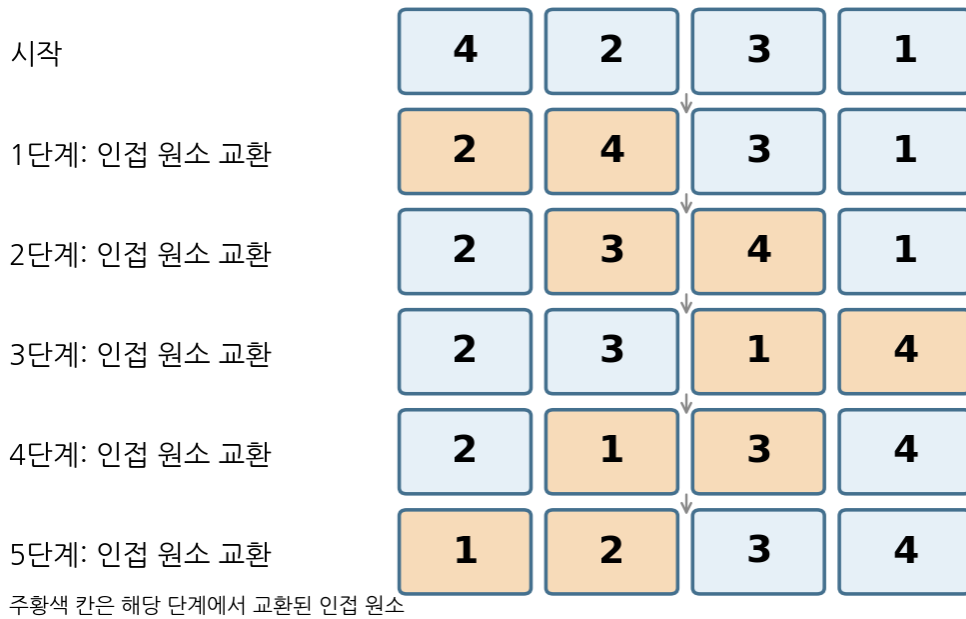
예: [4, 2, 3, 1]은 [4, 2]와 [3, 1]로 나뉘고 각각 [2, 4], [1, 3]이 된다. 두 구간의 앞을 비교하며 [1, 2, 3, 4]로 병합한다. 분할 깊이가 $O(\log n)$, 한 층의 병합 비용이 $O(n)$ 이므로 전체 시간은 $O(n \log n)$ 이다. tmp 배열 n 개와 재귀 호출 스택 $O(\log n)$ 을 사용하며, 전체 추가 공간은 $O(n)$ 이다. 이 구현은 정렬된 입력에서도 병합 및 복사를 수행한다.

2.3 Gnome sort: AI로 학습한 새 알고리즘

Gnome sort는 현재 위치 i 의 원소와 왼쪽 이웃을 비교한다. 두 원소가 이미 순서대로라면 i 를 한 칸 오른쪽으로 옮기고, 역순이면 둘을 교환하고 i 를 한 칸 왼쪽으로 옮긴다. 맨 앞에서 교환했으면 다음 위치로 진행한다. 삽입 정렬이 값을 저장한 뒤 여러 원소를 한 칸씩 밀어 넣는 데 비해 Gnome sort는 인접 교환을 반복해 값을 제자리로 보낸다.

[4, 2, 3, 1]에서는 처음 4와 2를 교환하여 [2, 4, 3, 1]이 된다. 4와 3도 교환해 [2, 3, 4, 1]을 만들고, 1이 왼쪽 이웃과 차례로 교환되면서 [1, 2, 3, 4]가 된다. 같은 키라면 교환하지 않으므로 안정 정렬이다. 이 구현은 지역변수 한 개만 사용해 추가 공간 $O(1)$ 이고 재귀도 없다. 이미 정렬된 자료에서는 $n-1$ 회 비교해 최선 $O(n)$, 일반·역순 자료에서는 앞뒤 이동과 교환이 반복되어 평균·최악 $O(n^2)$ 이다. 역순의 교환 횟수는 $n(n-1)/2$ 회이며, 이 코드의 이동 계수는 교환당 2 회다.

그림 2. Gnome sort의 단계별 진행: [4, 2, 3, 1]에서 이웃한 역순 쌍을 교환하며 왼쪽으로 돌아간다.



2.4 이론적 비교

알고리즘	최선 시간	평균 시간	최악 시간	추가 공간	안정성
삽입	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	예
병합	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	예
Gnome sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	예

삽입과 Gnome sort는 모두 정렬된 자료에 빠르고 제자리 안정 정렬이다. Gnome sort는 교환한 위치를 다시 방문하므로 같은 입력에서 삽입 정렬보다 비교 횟수가 많을 수 있다. 병합 정렬은 작업 배열을 사용하지만 입력 형태와 무관한 $O(n \log n)$ 상한을 제공한다.

3. 알고리즘 실험

3.1 입력과 측정 방법

환경은 C17, gcc -O2 -Wall -Wextra -pedantic 이다. 무작위 입력은 고정 시드 20260927의 xorshift32 생성기로 만들고, 형태는 무작위·정렬됨·역순·중복많음(키 0~7) 네 종류다. 크기는 1,000·2,000·4,000·8,000 개다. 동일한 source를 각 반복에 복사해 정렬을 5번 실행한다. clock()에 의한 5회 평균 시간(ms), 마지막 반복의 비교·이동 수, 원본 순서 위반 여부를 기록한다. 4×4×3 = 48 행이 results.csv (results.csv)에 있다.

지표	계산 방법	해석 시 주의
시간	정렬 호출 구간의 clock() 차이를 5회 평균	기기 부하·시계 해상도 영향
비교	키 비교 함수 호출 횟수	정렬 구현에 따라 호출 경로가 다름
이동	assign에 의한 item 배열 대입 횟수	지역변수 임시 저장과 memcpy 제외
안정성 관찰	같은 키에서 original의 역전 유무	동률이 없으면 판별 불가

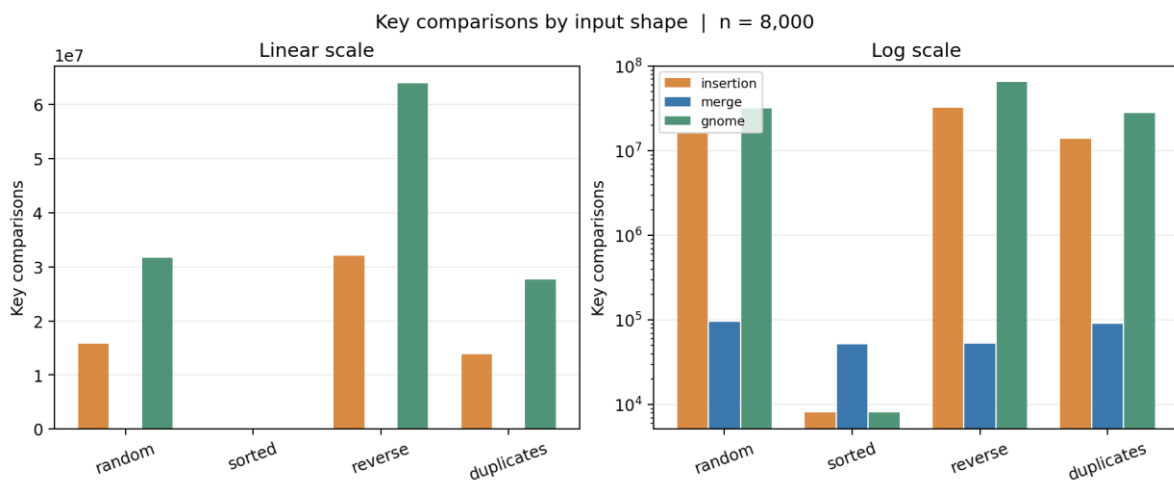
지표	계산 방법	해석 시 주의
추가 공간	코드의 작업 배열과 재귀를 분석	CSV의 직접 측정값은 아님

3.2 입력 유형별 결과 (n=8,000)

입력	정렬	평균 시간(ms)	비교	이동	재귀깊이
무작위	삽입	28.94740	15,834,821	15,834,824	1
무작위	병합	1.32520	93,589	207,616	14
무작위	Gnome	64.80320	31,661,643	31,653,666	1
정렬됨	삽입	0.01900	7,999	0	1
정렬됨	병합	0.34040	51,456	207,616	14
정렬됨	Gnome	0.00700	7,999	0	1
역순	삽입	56.15480	31,996,000	32,003,999	1
역순	병합	0.33960	52,352	207,616	14
역순	Gnome	125.89400	63,984,001	63,992,000	1
중복많음	삽입	24.42580	13,813,577	13,812,589	1
중복많음	병합	0.66360	89,892	207,616	14
중복많음	Gnome	58.08560	27,619,155	27,611,158	1

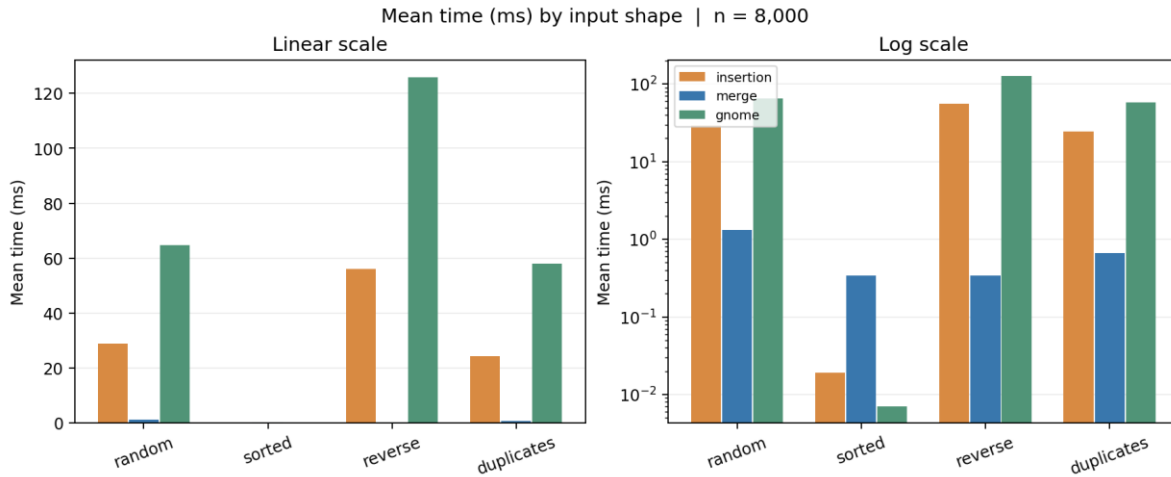
세 정렬 모두 같은 키의 상대 순서를 보존했다. 정렬된 입력에서 삽입과 Gnome sort는 각각 $n-1 = 7,999$ 회 비교하고 이동은 0 회다. 역순에서 삽입의 비교 횟수는 $n(n-1)/2 = 31,996,000$ 회이며, Gnome sort는 교환 뒤 왼쪽으로 돌아오며 추가 비교를 하여 63,984,001 회였다. Gnome sort의 이동 63,992,000회는 역전쌍 31,996,000개에 대해 인접 교환당 배열 대입 2회를 쓴 결과다. 병합의 이동 횟수는 이 크기에서 입력 형태에 관계없이 207,616 회였다.

그림 4. 입력 유형별 비교 횟수 (n=8,000; 선형·로그 축)



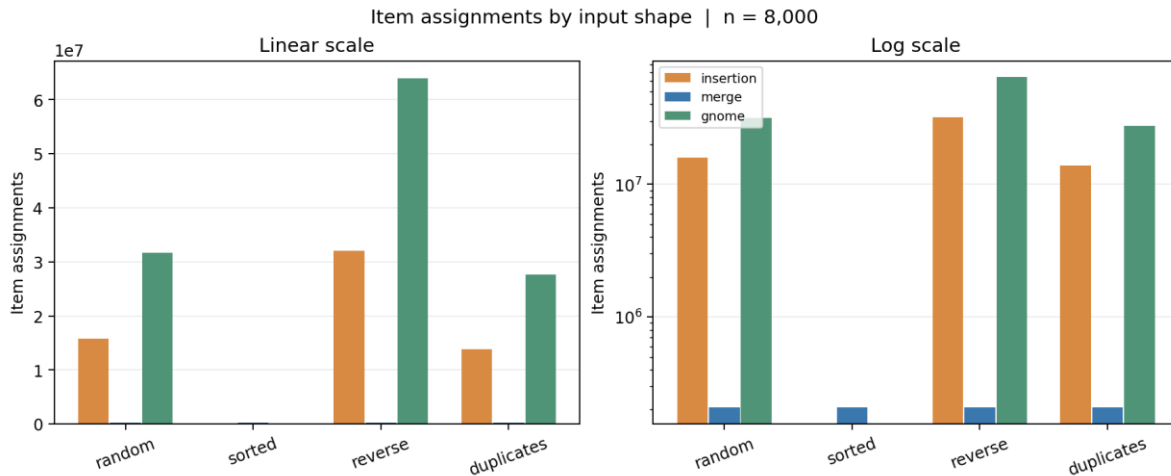
역순에서 Gnome sort 의 63,984,001 회는 삽입 정렬의 약 2 배다. 병합 정렬은 네 입력 모두 5 만~10 만 회 미만이며, 로그 축은 정렬된 입력의 작은 값도 함께 보여준다.

그림 5. 입력 유형별 평균 시간 (n=8,000; 선형·로그 축)



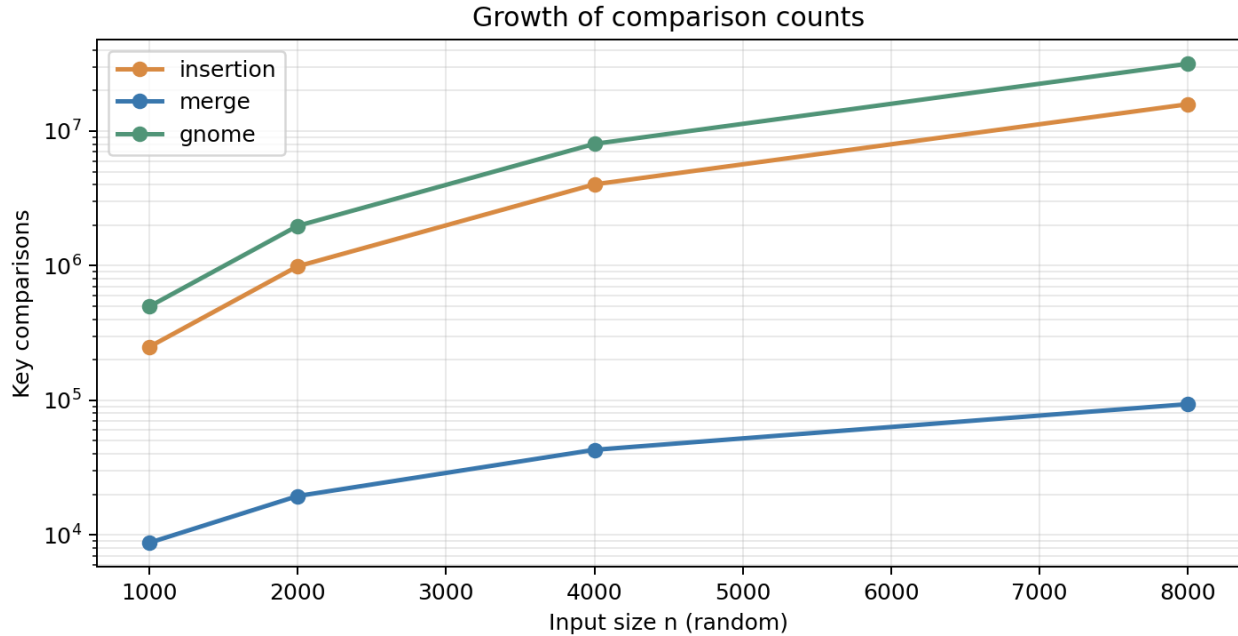
역순 Gnome sort 는 125.894 ms 로 가장 느리고 병합 정렬은 0.340 ms 다. 정렬된 입력에서는 삽입·Gnome sort 가 빠르다. 매우 짧은 시간은 측정 해상도에 민감하다.

그림 6. 입력 유형별 이동 횟수 (n=8,000; 선형·로그 축)



정렬된 입력에서 삽입·Gnome sort 의 이동은 0 회라 로그 축에는 표시되지 않는다. 역순에서는 Gnome sort 가 약 6,399 만 회로 삽입의 약 2 배이며, 병합의 이동은 입력 유형에 관계없이 207,616 회다.

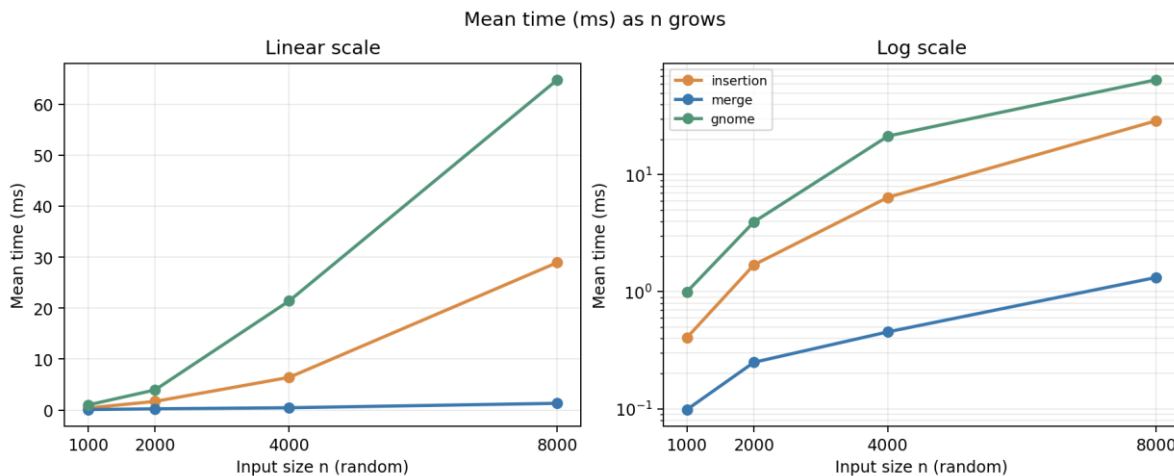
3.3 입력 크기에 따른 증가



무작위 입력에서 크기별 비교 횟수, 로그 세로축

세로축을 로그로 놓아 이차 정렬과 병합 정렬의 차이를 한 그림에서 읽는다. 무작위 입력 1,000→8,000 개에서 삽입 비교는 248,855→15,834,821(약 64 배), 병합은 8,727→93,589(약 10.7 배), Gnome sort 는 496,711→31,661,643(약 63.7 배)였다. 삽입과 Gnome sort 의 비교는 n 의 8 배 증가에 대하여 대략 64 배가 되고, 병합은 더 완만하다. 네 점만으로 점근 복잡도를 증명한 것은 아니다.

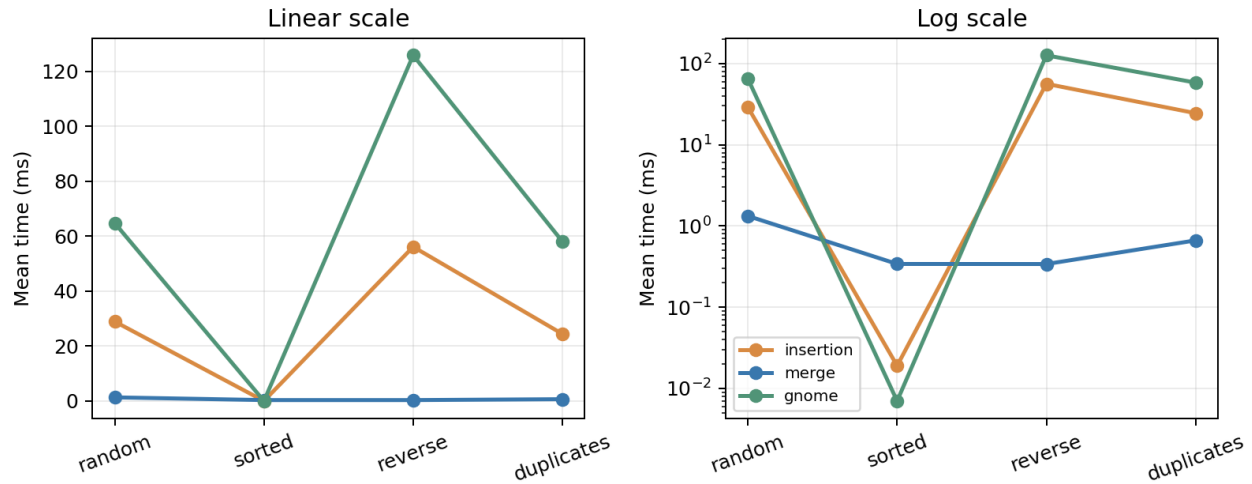
그림 8. 무작위 입력 크기별 평균 시간 ($n=1,000\sim8,000$; 선형·로그 축)



입력이 8 배가 되면 삽입·Gnome sort 의 시간은 이차 증가에 가까워지고 병합 정렬은 훨씬 완만하게 늘어난다. 선형 축은 절대 시간 차이, 로그 축은 세 곡선의 증가 추세를 함께 읽는 데 유용하다.

3.4 입력 모양에 따른 실행 시간

Time by input shape (n = 8,000; five repeats)



입력 모양별 평균 시간, 선형 축과 로그 축

왼쪽 선형 축에서는 Gnome sort의 역순 부담이 가장 크고 삽입 정렬도 뚜렷하게 증가한다. 오른쪽 로그 축에서는 매우 짧은 정렬된 입력의 삽입·Gnome sort 시간과 병합 시간을 함께 읽을 수 있다. 로그 축에서 막대·점의 높이 차이는 선형 축의 절대 차이가 아니다. 특히 0.01ms 정도의 매우 짧은 값은 clock() 분해능과 시스템 부하에 민감하다. 시간보다 비교 횟수가 입력 증가의 형태를 더 안정적으로 보여 준다.

그림 7의 비교 횟수와 그림 8의 평균 시간은 같은 무작위 입력을 1,000·2,000·4,000·8,000 개로 늘린 결과다. 비교 횟수는 삽입 248,855→15,834,821, 병합 8,727→93,589, Gnome sort 496,711→31,661,643 회로 변했다.

3.5 안정성과 메모리의 관계

삽입과 Gnome sort는 모두 추가 배열 없이 안정적으로 정렬하지만, 무작위·역순의 큰 입력에서는 이차 증가를 보인다. Gnome sort는 코드 흐름이 단순한 대신 한 원소를 왼쪽으로 옮길 때마다 교환 후 되돌아가므로 삽입 정렬보다 비교·배열 대입이 더 많다. 병합은 임시 Item n 개를 사용하는 대신 큰 입력에서 비교 수와 실행 시간이 훨씬 작았다. 작은 정렬된 자료에서는 두 제자리 정렬이 적합하고, 이 실험 범위의 큰 무작위 자료에서는 병합 정렬이 유리했다.

표 5. 추가 메모리·재귀깊이·안정성 (n=8,000)

알고리즘	추가 메모리	재귀깊이 (n=8,000)	안정성
삽입 정렬	O(1)	1 (재귀 없음)	안정
병합 정렬	O(n): Item 8,000 개 ≈ 64,000B + 호출 스택	14	안정
Gnome sort	O(1)	1 (재귀 없음)	안정

3.6 한계와 재현

시간 값은 한 실행 환경에서 5 회 평균한 결과다. 다른 컴파일러·CPU·부하에서는 순위와 배수가 달라질 수 있다. `clock()`은 벽시계 시간이 아니라 C 실행 환경에서 제공하는 프로세서 시간이며, 각 호출의 미세한 시간 차이는 해상도의 영향을 받는다. `xorshift32`의 시드는 고정되어 입력은 재현 가능하지만, 정렬 함수의 실행 시간은 재실행할 때 조금 달라진다. 비교·이동 횟수와 그래프는 저장된 `results.csv` (`results.csv`)에서 확인할 수 있다.